

B.Tech CSE

Software Engineering

Project Report

AI Review Intelligence System

Submitted by:

Nistha Agrawal
SAP ID: 590013402
Batch: B2 (Data Science)

Submitted to:

Prof. Mritunjay Kumar

Course Code: CSEG206_3
Semester: 4

Acknowledgement

I would like to express my sincere gratitude to **Prof. Mritunjay Kumar** for his guidance, support, and valuable insights throughout the development of this project. His suggestions and feedback played a significant role in shaping the overall direction and successful completion of this work.

I am also thankful to UPES for providing the necessary resources and academic environment that enabled me to carry out this project effectively.

I would also like to acknowledge the support of my peers and all those who contributed directly or indirectly to this project.

Abstract

The AI Review Intelligence System is a machine learning-based application developed to analyze textual reviews and automatically classify them into sentiment categories such as positive or negative. The system integrates a Streamlit-based user interface with a FastAPI backend and a TensorFlow-powered predictive model, forming a complete end-to-end pipeline for sentiment analysis.

The primary objective of this project is to provide an efficient and user-friendly solution for extracting meaningful insights from textual data. In addition to sentiment classification, the system offers features such as confidence scoring, basic emotion detection, and summary generation to enhance interpretability and usability.

This project demonstrates the practical implementation of software engineering principles, including modular design, system integration, and structured development methodologies, alongside machine learning techniques. The developed system can be applied in domains such as customer feedback analysis, product review evaluation, and decision support systems, thereby highlighting its real-world relevance and scalability.

Contents

Acknowledgement **1**

Abstract **2**

1 Introduction **6**

1.1 Background and Motivation 6
1.2 Objective of the Project 6
1.3 Scope of the System 6
1.4 Overview of the System 7
1.5 Significance of the Project 7

2 Literature Review **8**

2.1 Introduction 8
2.2 Traditional Approaches 8
2.3 Machine Learning Approaches 8
2.4 Deep Learning Approaches 8
2.5 Comparison of Approaches 9
2.6 Relevance to the Proposed System 9

3 Software Development Model **10**

3.1 Overview of SDLC 10
3.2 Waterfall Model 10
3.3 Agile Model 10
3.4 Agile Methodologies 11
3.5 Model Comparison 11
3.6 Selected Model Justification 11

4 System Analysis **12**

4.1 Problem Statement 12
4.2 Existing System 12
4.3 Proposed System 12
4.4 Feasibility Analysis 13

5 Requirements Engineering **14**

5.1 Introduction to Requirements 14
5.2 User Requirements 14
5.3 Functional Requirements 14
5.4 Non-Functional Requirements 15
5.5 Domain Requirements 16

5.6 External Interface Requirements 16
5.7 Design Constraints 16

6 System Design **17**
6.1 Overview 17
6.2 Design Principles 17
6.3 System Architecture 18
6.4 System Flowchart 19
6.5 Use Case Diagram 20
6.6 Sequence Diagram 21
6.7 Class Diagram 22
6.8 Data Flow Diagrams 23
6.8.1 DFD Level 0 23
6.8.2 DFD Level 1 24
6.8.3 DFD Level 2 25
6.9 User Interface Design 25

7 Methodology **26**
7.1 Overview 26
7.2 Dataset Description 26
7.3 Data Preprocessing 26
7.4 Model Architecture 27
7.5 Prediction Pipeline 27
7.6 Tools and Technologies Used 27

8 Implementation **28**
8.1 Overview 28
8.2 Technologies Used 28
8.3 System Implementation 28
8.4 Working of the System 28
8.5 Model Integration 29
8.6 Code Structure 29
8.7 Code Snippets 29

9 Testing **30**
9.1 Testing Approach 30
9.2 Unit Testing 30
9.3 Integration Testing 30
9.4 System Testing 31
9.5 Test Case Design Techniques 31

9.6 Test Cases 31

10 Results and Discussion **33**
10.1 Overview 33
10.2 Model Performance 33
10.3 System Outputs 33
10.4 Sample Output Screenshots 34
10.5 Observations 35
10.6 Discussion 35

11 Conclusion **36**

12 Future Scope **37**

13 References **38**

14 Appendix **39**
14.1 Use of AI Tools 39
14.2 Tools and Technologies 39
14.3 Project Structure 39
14.4 Application Access 39
14.5 System Screenshots 40
14.5.1 Application Output 40
14.5.2 API Interface 41

1 Introduction

1.1 Background and Motivation

In the current digital era, a vast amount of textual data is generated through online platforms such as e-commerce websites, social media, and customer feedback systems. These reviews contain valuable insights about user opinions and experiences. However, manually analyzing such large volumes of data is time-consuming, inefficient, and prone to inconsistencies.

Sentiment analysis provides an automated approach to interpret and classify textual data based on the underlying emotions or opinions. This has become an essential tool for organizations to understand customer behavior, improve products, and support decision-making processes.

1.2 Objective of the Project

The primary objective of the AI Review Intelligence System is to develop an intelligent and efficient solution for analyzing textual reviews using machine learning techniques. The system aims to:

- Classify user reviews into sentiment categories such as positive or negative
- Provide confidence scores associated with predictions
- Generate a basic emotional interpretation of the input text
- Produce a concise summary for quick understanding

1.3 Scope of the System

The system is designed as a web-based application that integrates a user-friendly frontend with a backend API and a machine learning model. It allows users to input textual data and receive real-time sentiment analysis results.

The scope of the system includes applications such as customer feedback analysis, product review evaluation, and opinion mining. However, the system is limited to binary sentiment classification and is dependent on the training dataset for accuracy.

1.4 Overview of the System

The AI Review Intelligence System follows a modular architecture consisting of three main components:

- **Frontend:** Developed using Streamlit, providing an interactive interface for user input and visualization.
- **Backend:** Implemented using FastAPI, responsible for handling requests and processing data.
- **Machine Learning Model:** A TensorFlow-based model trained on the IMDB dataset for sentiment prediction.

The system operates in a client-server model where user input is processed through the backend and analyzed by the machine learning model, and the results are returned to the user interface.

1.5 Significance of the Project

This project demonstrates the integration of software engineering principles with machine learning techniques to build a scalable and efficient system. It highlights the importance of modular design, system integration, and structured development methodologies in solving real-world problems.

The system can be further extended to support multi-language processing, advanced emotion detection, and large-scale deployment, making it a practical solution for real-world applications.

2 Literature Review

2.1 Introduction

Sentiment analysis has emerged as a significant area of research in Natural Language Processing (NLP), focusing on extracting opinions and emotions from textual data. With the rapid growth of online platforms, analyzing user-generated content such as reviews and feedback has become increasingly important for businesses and decision-making systems.

2.2 Traditional Approaches

Early approaches to sentiment analysis relied on rule-based and lexicon-based methods. These techniques use predefined dictionaries of words associated with positive or negative sentiments to classify text.

While these methods are simple and interpretable, they suffer from several limitations:

- Inability to handle context and sarcasm
- Limited adaptability to new domains
- Dependence on manually curated dictionaries

2.3 Machine Learning Approaches

Machine learning techniques improved sentiment analysis by enabling models to learn patterns from data. Algorithms such as Naive Bayes, Support Vector Machines (SVM), and Logistic Regression have been widely used for classification tasks.

These methods provide better accuracy compared to traditional approaches; however, they require feature engineering and may not capture complex linguistic structures effectively.

2.4 Deep Learning Approaches

Recent advancements in deep learning have significantly enhanced sentiment analysis capabilities. Models such as Recurrent Neural Networks (RNN), Long Short-Term Memory (LSTM), and transformer-based architectures can capture contextual relationships within text.

These models automatically learn features from data, reducing the need for manual feature engineering and improving prediction performance.

2.5 Comparison of Approaches

- Traditional methods are simple but lack contextual understanding
- Machine learning models improve accuracy but require feature engineering
- Deep learning models provide higher accuracy and better contextual understanding

2.6 Relevance to the Proposed System

The AI Review Intelligence System utilizes a deep learning-based approach for sentiment analysis. This enables the system to achieve improved accuracy and handle variations in textual data more effectively.

Compared to traditional and basic machine learning methods, the chosen approach provides better performance and scalability, making it suitable for real-world applications such as customer feedback analysis and review classification.

3 Software Development Model

3.1 Overview of SDLC

The Software Development Life Cycle (SDLC) provides a structured approach for developing software systems through well-defined phases such as planning, design, implementation, testing, and deployment. It ensures systematic development, better quality, and efficient project management.

In this project, SDLC principles were followed to organize the development of the AI Review Intelligence System in a structured and iterative manner.

3.2 Waterfall Model

The Waterfall model is a linear and sequential software development approach where each phase must be completed before moving to the next. It is simple and easy to understand but lacks flexibility when requirements change.

While the Waterfall model is suitable for projects with well-defined and stable requirements, it is less effective for projects involving experimentation and iterative improvements, such as machine learning-based systems.

3.3 Agile Model

The Agile model is an iterative and incremental approach that emphasizes flexibility, continuous improvement, and collaboration. Development is carried out in small cycles, allowing frequent updates and refinements based on feedback.

In the context of this project, Agile methodology was adopted to allow iterative development of the system components such as the user interface, backend API, and machine learning model. This approach enabled continuous testing and improvement of model performance and system functionality.

3.4 Agile Methodologies

Agile development includes several methodologies such as Scrum, Extreme Programming (XP), Lean, and Kanban:

- **Scrum:** Focuses on iterative development using short cycles called sprints.
- **Extreme Programming (XP):** Emphasizes code quality and frequent releases.
- **Lean:** Aims at minimizing waste and maximizing efficiency.
- **Kanban:** Focuses on visualizing workflow and improving process efficiency.

Although a formal Scrum process was not strictly followed, the project development loosely incorporated Agile principles such as incremental development, continuous testing, and iterative refinement.

3.5 Model Comparison

- The Waterfall model provides a structured approach but lacks flexibility.
- Agile allows adaptability and iterative improvement, making it suitable for dynamic projects.
- Machine learning-based systems benefit more from Agile due to the need for experimentation and tuning.

3.6 Selected Model Justification

The Agile model was chosen for the development of the AI Review Intelligence System due to its flexibility and iterative nature. The project involved continuous experimentation with model performance, UI improvements, and backend integration, which required frequent modifications.

Agile methodology enabled the development process to adapt to changes efficiently, ensuring better performance, usability, and overall system quality.

4 System Analysis

4.1 Problem Statement

With the rapid growth of online platforms, a large volume of textual reviews is generated daily by users. These reviews contain valuable insights regarding user opinions, product quality, and service satisfaction. However, manually analyzing such data is inefficient, time-consuming, and prone to human error.

There is a need for an automated system that can efficiently process textual data, accurately classify sentiments, and present results in an understandable format. The absence of such systems limits the ability of organizations to make data-driven decisions based on user feedback.

The AI Review Intelligence System aims to address this problem by providing a machine learning-based solution for automated sentiment analysis.

4.2 Existing System

Existing approaches to sentiment analysis include manual analysis, rule-based systems, and basic machine learning models.

- **Manual Analysis:** Requires human effort and is not scalable for large datasets.
- **Rule-Based Systems:** Depend on predefined sentiment lexicons and fail to capture context, sarcasm, and complex language patterns.
- **Basic Machine Learning Models:** Provide better accuracy but require extensive feature engineering and may not fully capture contextual relationships in text.

These systems often lack real-time processing capabilities, user-friendly interfaces, and integrated visualization, making them less effective for practical applications.

4.3 Proposed System

The AI Review Intelligence System is designed as an integrated solution that combines a user-friendly interface, a backend processing system, and a machine learning model for sentiment analysis.

- The system allows users to input textual reviews through an interactive interface.
- The backend processes the input using FastAPI and performs preprocessing operations such as cleaning and tokenization.
- A TensorFlow-based machine learning model analyzes the processed data and predicts sentiment along with a confidence score.
- The results are displayed to the user along with additional features such as basic emotion detection and summary generation.

Compared to existing systems, the proposed system provides improved accuracy, real-time analysis, and better usability, making it suitable for real-world applications such as customer feedback analysis and review classification.

4.4 Feasibility Analysis

Technical Feasibility: The system is implemented using widely used technologies such as Python, Streamlit, FastAPI, and TensorFlow, ensuring ease of development and deployment.

Economic Feasibility: The project does not require expensive resources, as it is developed using open-source tools and publicly available datasets.

Operational Feasibility: The system is user-friendly and requires minimal technical knowledge, making it suitable for general users.

Time Feasibility: The project can be completed within the given academic timeline due to its modular design and use of efficient development tools.

5 Requirements Engineering

5.1 Introduction to Requirements

Requirements define the expected functionality and constraints of the system. This section specifies the requirements of the AI Review Intelligence System following the IEEE 830 standard, ensuring clarity, completeness, and consistency.

5.2 User Requirements

User requirements describe the functionalities from the end-user perspective.

- The system should allow users to input textual reviews.
- The system should display sentiment results in an understandable format.
- The system should provide a confidence score for predictions.
- The system should display additional insights such as emotion and summary.
- The system should provide a simple and user-friendly interface.

5.3 Functional Requirements

Functional requirements define the core functionalities of the system.

ID	Requirement	Description
FR1	Input Handling	The system shall accept textual input from the user through the interface.
FR2	Text Processing	The system shall preprocess input text by cleaning and tokenizing it.
FR3	Sentiment Prediction	The system shall classify input text as positive or negative using a trained model.
FR4	Confidence Score	The system shall display a probability score associated with the prediction.
FR5	Emotion Detection	The system shall provide a basic emotion interpretation of the input text.

FR6	Summary Generation	The system shall generate a brief summary of the input text.
FR7	API Communication	The frontend shall communicate with the backend using API requests.
FR8	Result Display	The system shall display results clearly on the user interface.

5.4 Non-Functional Requirements

Non-functional requirements define the quality attributes of the system.

ID	Category	Description
NFR1	Performance	The system shall provide results within 2 seconds under normal conditions.
NFR2	Reliability	The system shall handle invalid or empty inputs without failure.
NFR3	Usability	The interface shall be simple and easy to use for non-technical users.
NFR4	Scalability	The system shall support future enhancements and increased user load.
NFR5	Maintainability	The system shall be modular to allow easy updates.
NFR6	Portability	The system shall run on standard web browsers and systems supporting Python.
NFR7	Security	The system shall safely handle user input and prevent misuse.
NFR8	Accuracy	The system shall maintain high prediction accuracy based on the trained model.

5.5 Domain Requirements

Domain requirements are specific to the application domain of sentiment analysis.

- The system must process natural language text inputs.
- The system must use a trained dataset (IMDB dataset) for sentiment classification.
- The system must convert textual data into numerical form for model processing.

5.6 External Interface Requirements

User Interface:

- The system provides a web-based interface using Streamlit.
- Users can enter text and view results interactively.

Software Interface:

- The frontend communicates with the backend via FastAPI.
- The backend interacts with the machine learning model for predictions.

5.7 Design Constraints

- The system must be implemented using Python-based technologies.
- The model performance depends on the dataset used for training.
- The system is designed for binary sentiment classification.

6 System Design

6.1 Overview

The system design of the AI Review Intelligence System focuses on creating a modular, scalable, and efficient architecture that integrates the frontend, backend, and machine learning components. The design ensures smooth data flow, clear separation of responsibilities, and ease of maintenance.

6.2 Design Principles

The system is developed using fundamental software design principles:

- **Abstraction:** Complex operations such as model prediction and preprocessing are hidden behind simple interfaces.
- **Modularity:** The system is divided into independent modules including frontend, backend, and machine learning model.
- **Cohesion:** Each module performs a specific task, ensuring high cohesion within components.
- **Coupling:** Loose coupling is maintained between modules through API-based communication.
- **Refinement:** The system is developed incrementally, refining components over time.

6.3 System Architecture

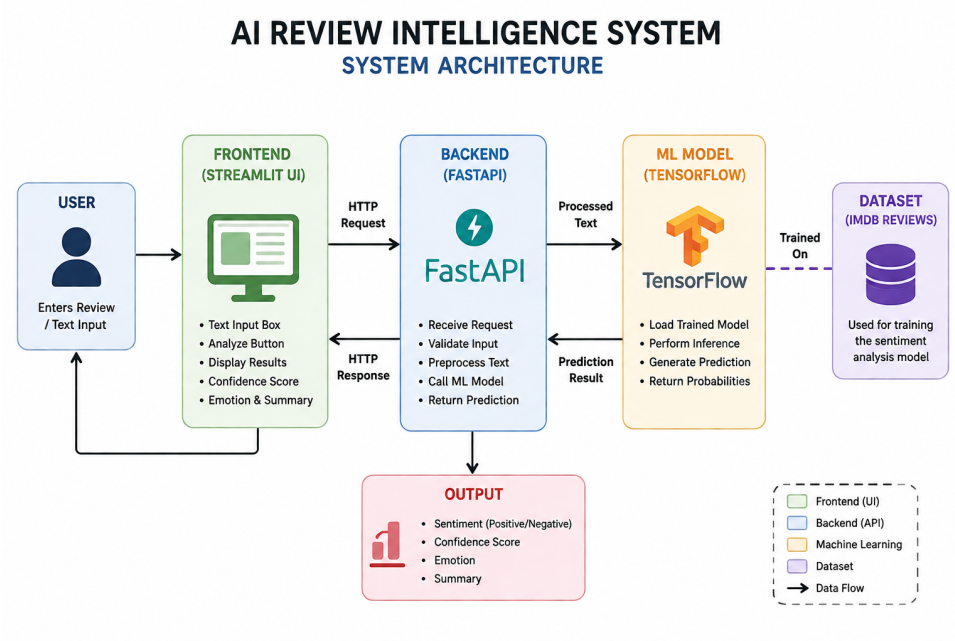


Figure 1: System Architecture

The architecture follows a client-server model integrating Streamlit UI, FastAPI backend, and a TensorFlow-based machine learning model.

6.4 System Flowchart

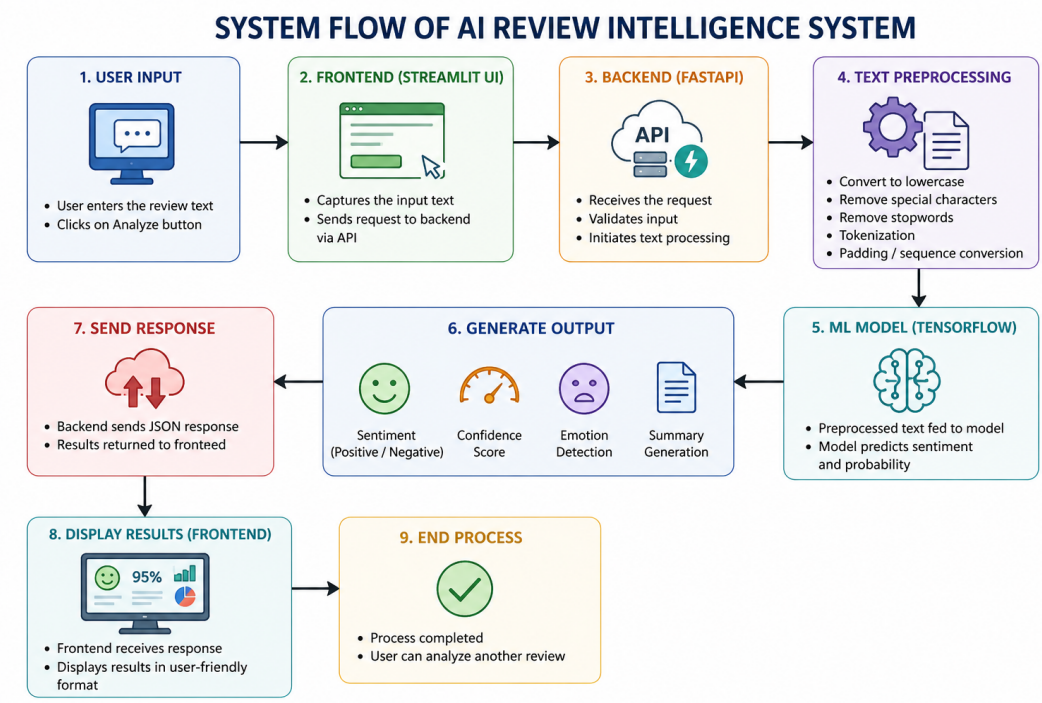


Figure 2: System Flowchart

The flowchart illustrates the complete workflow from user input to final result generation.

6.5 Use Case Diagram

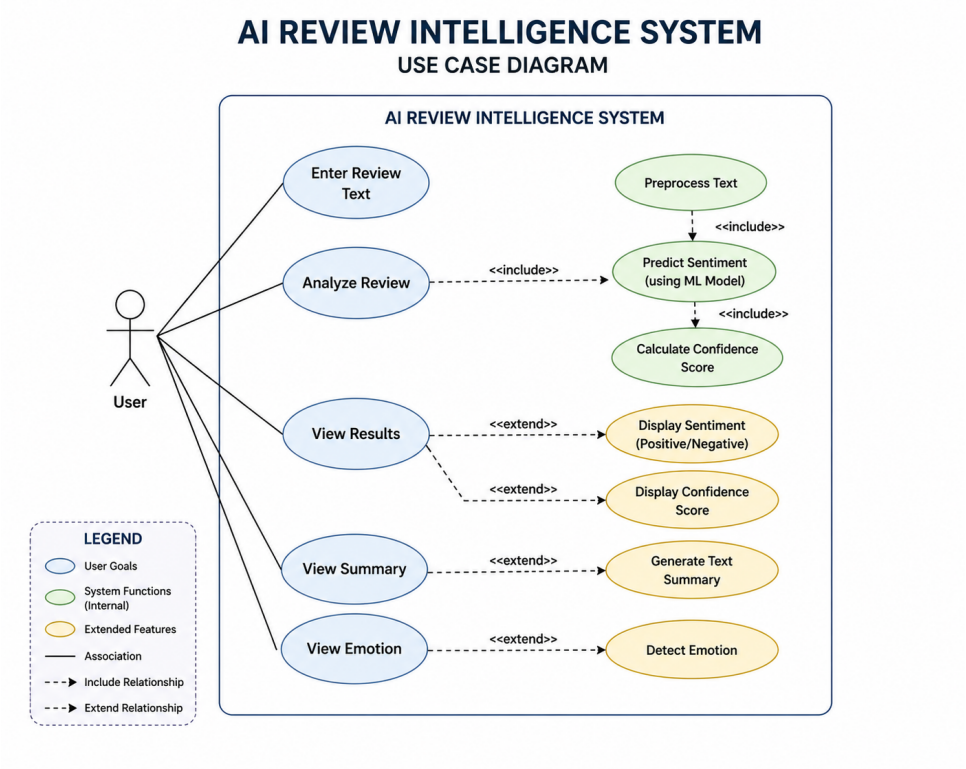


Figure 3: Use Case Diagram

The diagram represents interactions between the user and system functionalities such as input, analysis, and result viewing.

7 Methodology

7.1 Overview

The methodology of the AI Review Intelligence System describes the complete pipeline followed for transforming raw textual input into meaningful sentiment predictions. The system integrates data preprocessing, machine learning model inference, and result generation in a structured manner.

7.2 Dataset Description

The system uses the IMDB movie review dataset, which consists of labeled textual reviews categorized as positive or negative. This dataset is widely used for sentiment analysis tasks and provides a balanced distribution of sentiment classes.

The dataset is utilized to train the machine learning model, enabling it to learn patterns and relationships between textual features and sentiment labels.

7.3 Data Preprocessing

Data preprocessing is a crucial step to convert raw text into a format suitable for machine learning models. The following steps are performed:

- **Text Cleaning:** Removal of special characters, punctuation, and irrelevant symbols.
- **Lowercasing:** Conversion of all text to lowercase to maintain consistency.
- **Tokenization:** Splitting text into individual words or tokens.
- **Sequence Conversion:** Mapping words to numerical indices using a predefined word index.
- **Padding and Truncation:** Ensuring all sequences are of equal length for model input.

These preprocessing steps ensure that the input data is standardized and suitable for model prediction.

7.4 Model Architecture

The system uses a TensorFlow-based deep learning model trained on the IMDB dataset. The model is designed to capture patterns in textual data and classify reviews into sentiment categories.

The trained model is stored and loaded during runtime, enabling real-time prediction without retraining.

7.5 Prediction Pipeline

The prediction pipeline follows a sequence of steps:

1. User enters a textual review through the Streamlit interface.
2. The input is sent to the FastAPI backend via an API request.
3. The backend preprocesses the input text using cleaning, tokenization, and sequence conversion.
4. The processed input is passed to the trained machine learning model.
5. The model generates a probability score representing sentiment.
6. Based on the probability, the sentiment is classified as positive or negative.
7. Additional outputs such as confidence score, emotion, and summary are generated.
8. The final results are returned to the frontend and displayed to the user.

7.6 Tools and Technologies Used

- **Python:** Core programming language
- **Streamlit:** Frontend interface
- **FastAPI:** Backend API handling
- **TensorFlow/Keras:** Machine learning model development
- **NumPy:** Numerical computations

8 Implementation

8.1 Overview

The implementation of the AI Review Intelligence System involves integrating the frontend interface, backend API, and machine learning model to create a functional end-to-end application.

8.2 Technologies Used

- **Python:** Core programming language
- **Streamlit:** User interface development
- **FastAPI:** Backend API development
- **TensorFlow/Keras:** Model loading and prediction

8.3 System Implementation

The system is divided into three main components:

- **Frontend:** Built using Streamlit, it allows users to input text and view results.
- **Backend:** Developed using FastAPI, it handles API requests and processes input data.
- **Model:** A pre-trained TensorFlow model is loaded to perform sentiment prediction.

8.4 Working of the System

1. The user enters a review in the Streamlit interface.
2. The frontend sends a POST request to the FastAPI backend.
3. The backend receives the request and preprocesses the input text.
4. The processed input is passed to the trained model.
5. The model predicts sentiment and returns a probability score.
6. The backend sends the response to the frontend.
7. The frontend displays sentiment, confidence score, emotion, and summary.

8.5 Model Integration

The machine learning model is trained using the IMDB dataset and saved as a `.h5` file. During runtime, the model is loaded once and reused for predictions, ensuring efficient performance.

8.6 Code Structure

The project is organized into multiple modules for better maintainability:

- `app.py` – Streamlit frontend
- `main.py` – FastAPI backend
- `predict.py` – Prediction logic
- `model_files/` – Stored model and word index

8.7 Code Snippets

API Endpoint (FastAPI):

```
@app.post("/predict")
def get_sentiment(review: Review):
    return predict(review.text)
```

Prediction Function:

```
prob = float(model.predict(seq)[0][0])
label = "Positive" if prob >= 0.5 else "Negative"
```

Frontend Request:

```
res = requests.post(API_URL, json={"text": text})
```

9 Testing

Testing is performed to ensure that the system meets both functional and non-functional requirements and operates correctly under various conditions.

9.1 Testing Approach

The system is tested using multiple testing techniques:

- **Black Box Testing:** Verifies system functionality without knowledge of internal code.
- **White Box Testing:** Tests internal logic such as preprocessing and prediction functions.
- **Functional Testing:** Ensures all features work as expected.
- **Validation Testing:** Checks whether the system meets user requirements.

9.2 Unit Testing

Unit testing is performed on individual components such as:

- Text preprocessing functions (cleaning, tokenization)
- Prediction function
- API endpoint handling

Each module is tested independently to ensure correctness before integration.

9.3 Integration Testing

Integration testing verifies the interaction between different components:

- Communication between Streamlit frontend and FastAPI backend
- Data flow from API to machine learning model
- Response handling and display on UI

This ensures that all modules work together seamlessly.

9.4 System Testing

System testing is performed on the complete system to validate end-to-end functionality.

- Input is provided through the user interface
- Output is verified for correctness and consistency
- Error handling is tested for invalid inputs

9.5 Test Case Design Techniques

The following techniques are used for designing test cases:

- **Equivalence Partitioning:** Inputs are divided into valid and invalid groups.
- **Boundary Value Analysis:** Testing edge cases such as empty input.
- **Decision-Based Testing:** Verifying classification outcomes (positive/negative).

9.6 Test Cases

ID	Test Description	Input	Expected Output	Status
TC1	Valid Positive Review	"This product is amazing"	Positive sentiment with high confidence	Pass
TC2	Valid Negative Review	"Worst experience ever"	Negative sentiment with high confidence	Pass
TC3	Neutral Text	"The product is okay"	Moderate or neutral-like prediction	Pass
TC4	Empty Input	""	Error message displayed	Pass
TC5	Special Characters Input	"@@@###"	Handled without crash, safe output	Pass

ID	Test Description	Input	Expected Output	Status
TC6	Long Input Text	Large paragraph review	Correct prediction without delay	Pass
TC7	API Failure Case	Backend not running	Error message shown to user	Pass
TC8	Mixed Sentiment	"Good but could be better"	Balanced or slightly positive prediction	Pass

10 Results and Discussion

10.1 Overview

This section presents the outcomes of the AI Review Intelligence System along with observations regarding system performance, accuracy, and usability. The results demonstrate the effectiveness of the machine learning model and the overall system design.

10.2 Model Performance

The sentiment analysis model is trained on the IMDB dataset and achieves high accuracy in classifying reviews as positive or negative. The model performs well on standard inputs and is capable of identifying sentiment patterns in textual data.

In addition to accuracy, evaluation metrics such as consistency of predictions and response time indicate that the system performs efficiently under normal operating conditions.

10.3 System Outputs

The system generates the following outputs for a given input:

- Sentiment classification (Positive/Negative)
- Confidence score represented as a percentage
- Basic emotion interpretation
- Summary of the input text

10.4 Sample Output Screenshots



Figure 9: Positive Sentiment Output

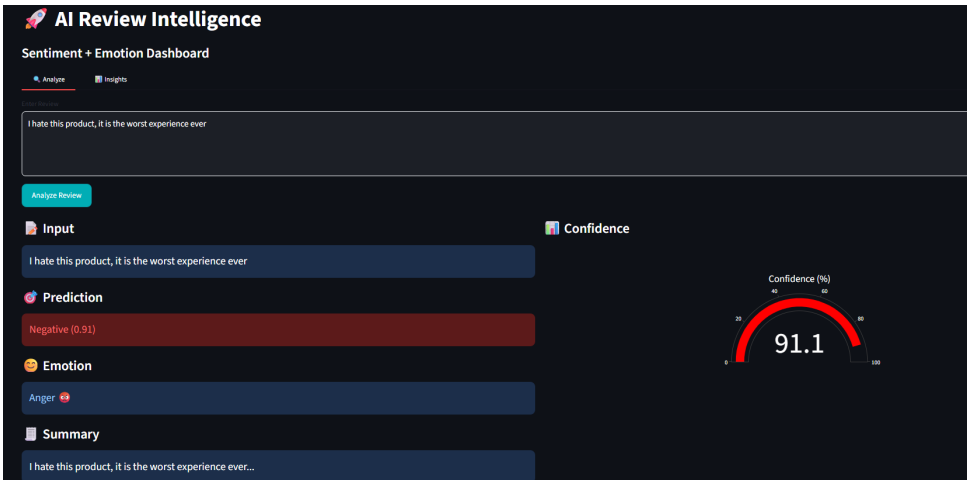


Figure 10: Negative Sentiment Output

The screenshots illustrate the system's ability to process user input and display results in a clear and user-friendly manner.

10.5 Observations

- The system provides accurate predictions for clearly positive and negative inputs.
- It performs well in real-time with minimal delay in response.
- The user interface enhances usability by presenting results in an intuitive format.
- The system handles edge cases such as empty input and invalid data gracefully.
- Slight variations may occur for ambiguous or mixed sentiment inputs, which is expected in natural language processing systems.

10.6 Discussion

The results indicate that the integration of machine learning with a modular software architecture leads to an efficient and scalable solution for sentiment analysis. The system successfully combines usability, performance, and accuracy.

While the current implementation focuses on binary sentiment classification, the system can be further extended to support multi-class sentiment analysis and advanced emotion detection.

11 Conclusion

The AI Review Intelligence System successfully demonstrates the application of machine learning and software engineering principles in building an efficient and user-friendly sentiment analysis platform. The system integrates a Streamlit-based frontend, a FastAPI backend, and a TensorFlow-based predictive model to deliver real-time sentiment classification.

The developed system is capable of analyzing textual reviews, generating sentiment predictions, and providing additional insights such as confidence scores, emotion detection, and summary generation. The modular architecture ensures scalability, maintainability, and ease of future enhancements.

Through this project, key concepts such as system design, requirement analysis, testing methodologies, and model integration have been effectively implemented. The results indicate that the system performs reliably under various input conditions and provides meaningful outputs for practical applications.

Overall, the project highlights the effectiveness of combining machine learning techniques with structured software development approaches to solve real-world problems such as automated sentiment analysis.

12 Future Scope

The AI Review Intelligence System can be further enhanced in several ways to improve its functionality, accuracy, and scalability:

- **Multi-language Support:** Extending the system to analyze reviews in multiple languages to increase usability across diverse user groups.
- **Advanced Emotion Detection:** Incorporating deep learning models capable of detecting a wider range of emotions beyond basic sentiment classification.
- **Multi-class Sentiment Analysis:** Expanding the model to classify sentiments into categories such as positive, negative, and neutral for more detailed analysis.
- **Model Optimization:** Improving model performance using advanced architectures such as transformers and fine-tuning techniques.
- **Real-time Data Integration:** Integrating the system with live data sources such as social media platforms or review websites.
- **Cloud Deployment:** Deploying the system on cloud platforms to support scalability and handle larger user traffic.
- **Enhanced Visualization:** Adding dashboards and analytics for better visualization of sentiment trends and insights.

13 References

- IMDB Dataset. Available at: `https://ai.stanford.edu/~amaas/data/sentiment/`
- TensorFlow Documentation. Available at: `https://www.tensorflow.org/`
- FastAPI Documentation. Available at: `https://fastapi.tiangolo.com/`
- Streamlit Documentation. Available at: `https://docs.streamlit.io/`
- Jurafsky, D., & Martin, J. H. (2023). *Speech and Language Processing.* (For NLP concepts)
- Han, J., Kamber, M., & Pei, J. (2011). *Data Mining: Concepts and Techniques.*

14 Appendix

14.1 Use of AI Tools

AI-based tools were used to assist in generating system diagrams and enhancing visualization quality. The conceptual design, system architecture, implementation logic, and overall development of the project were independently carried out.

14.2 Tools and Technologies

The following tools and technologies were used for the development of the system:

- **Python** – Core programming language
- **Streamlit** – Development of user interface
- **FastAPI** – Backend API framework
- **TensorFlow/Keras** – Machine learning model development
- **NumPy** – Numerical computations
- **Plotly** – Visualization of confidence metrics

14.3 Project Structure

The project is organized into modular components for better maintainability and scalability:

- `app.py` – Streamlit frontend application
- `main.py` – FastAPI backend
- `predict.py` – Prediction and preprocessing logic
- `model_files/` – Contains trained model and word index

14.4 Application Access

The user interface of the system can be accessed locally using the following link:

`http://localhost:8501/`

The backend API documentation is available at:

`http://127.0.0.1:8000/docs`

14.5 System Screenshots

14.5.1 Application Output

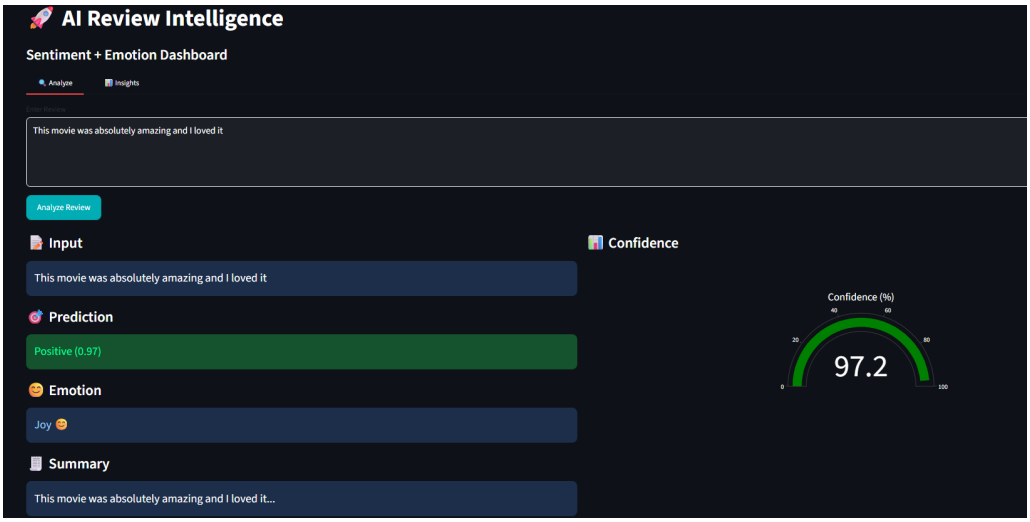


Figure 11: Output Interface Showing Sentiment Prediction, Confidence Meter, and Emotion

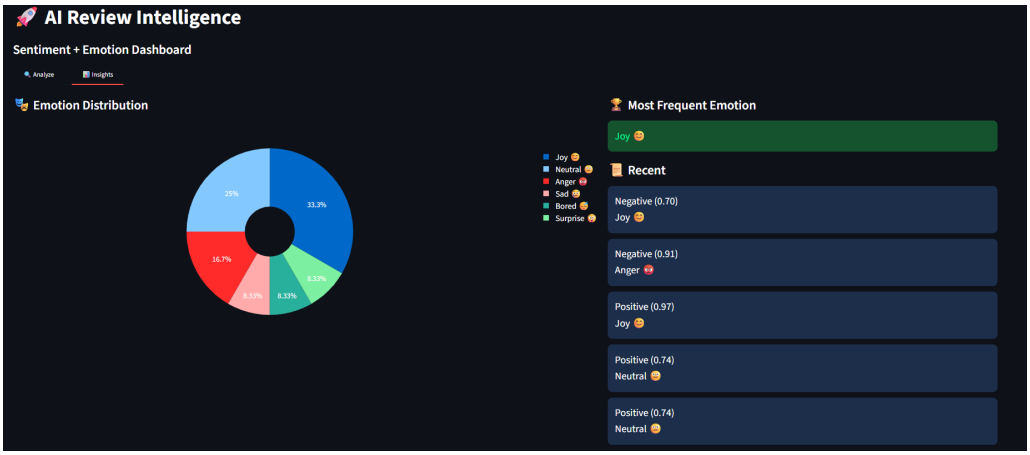


Figure 12: Insights Dashboard Showing Emotion Distribution and Prediction History

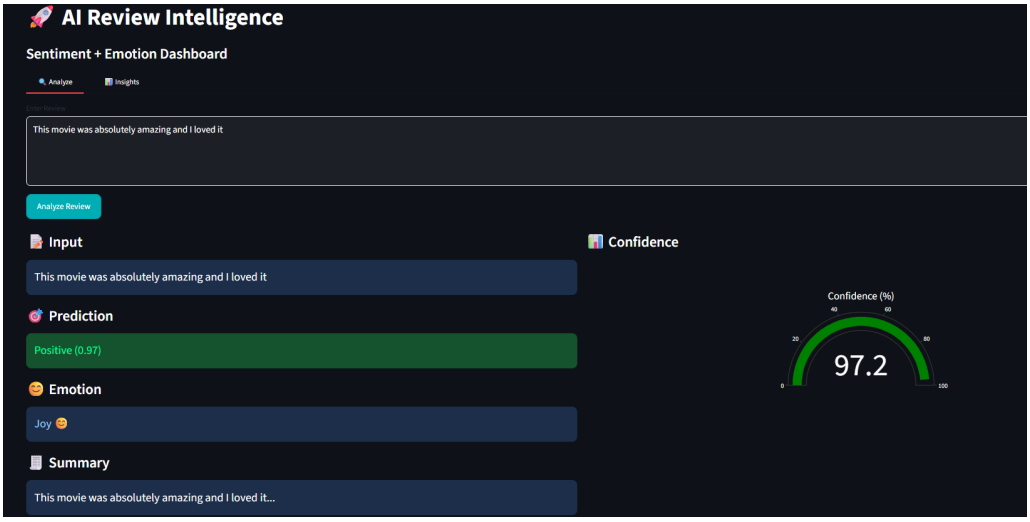


Figure 13: Output Interface Showing Sentiment Prediction, Confidence Meter, Emotion, and Summary

14.5.2 API Interface

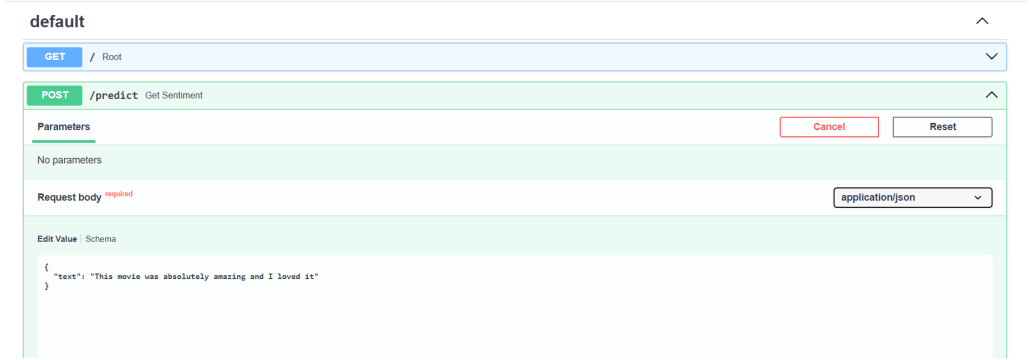


Figure 14: FastAPI Interactive Documentation Interface

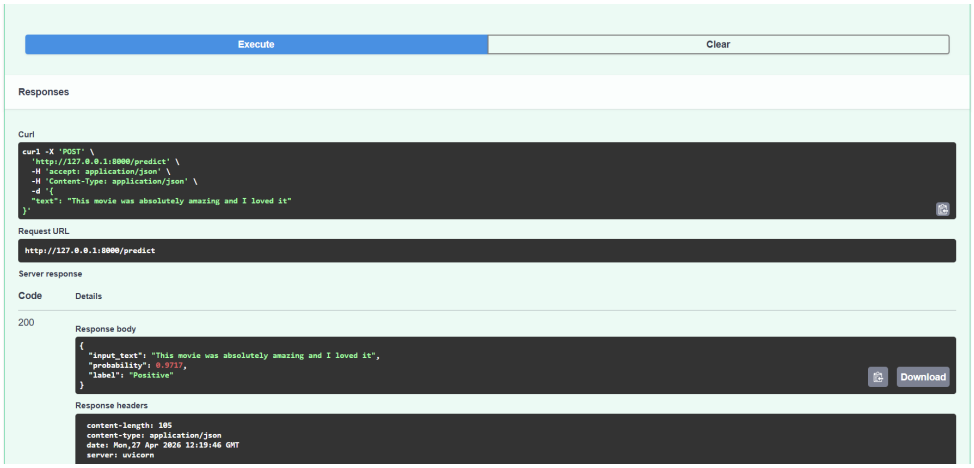


Figure 15: Sample API Response in JSON Format